

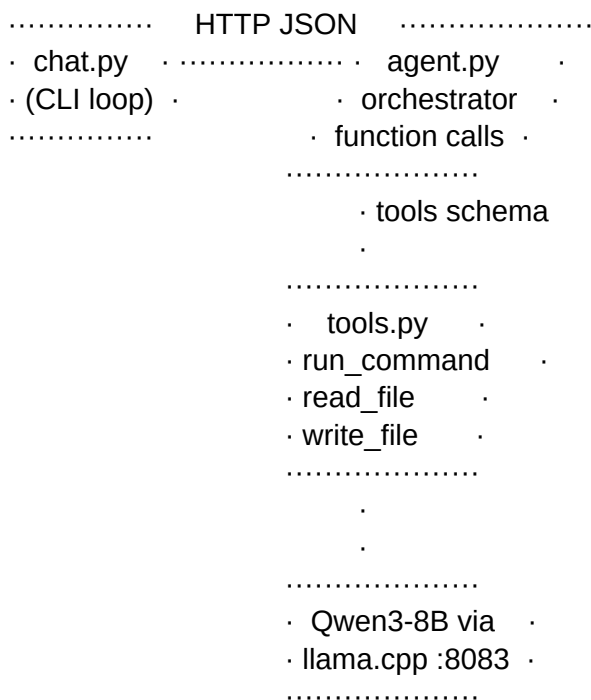
aios-agent v2.1 · SRE Agent with native function calling

A lightweight Site Reliability Engineering (SRE) agent that uses native function calling on top of the local Qwen3-8B model (served by llama.cpp). It can run Linux commands, read configuration/log files, and write controlled changes, all through a conversation in Spanish.

What does it do?

- Answers sysadmin questions in Spanish.
- Executes shell commands on the local machine (`run\_command`).
- Reads configuration files and logs (`read\_file`).
- Writes files to allowed paths, blocking system directories (`write\_file`).
- Maintains conversational context and performs up to 5 reasoning turns of tool·LLM.
- Plans and executes multi-step tasks without intermediate human intervention.

Architecture



- `chat.py`: interactive loop.
- `agent.py`: manages messages, calls the LLM, executes tool calls, and returns responses.
- `tools.py`: tool definitions and handlers.

Roadmap

Features implemented in v2.0 (all completed):

- Native function calling over llama.cpp server (`/v1/chat/completions`)
- Tool `run\_command`: execute shell commands with timeout, capturing stdout/stderr/exit_code/elapsed
- Tool `read\_file`: read files with permission checks and size limits
- Tool `write\_file`: write files, blocking critical system paths
- Conversational loop with up to 5 tool·LLM turns and persistent per-session context

- Multi-step planning and execution without intermediate human intervention
- Basic security: warning before destructive commands and protection of `/etc`, `/boot`, `/sys`, `/proc`, `/dev`
- Interactive CLI in Spanish (`chat.py`) with commands `salir`/`exit`/`quit`
- Complete README.md with architecture, usage, and roadmap
- Executive documentation in PDF (`docs/ejecutivo.pdf`)

Multi-step planning

For complex tasks the agent does not perform a single function call: it first thinks, decomposes, and then executes the steps sequentially.

How it works

1. Planning prompt: the system prompt instructs the LLM to generate a numbered plan of steps and keep executing it with the instruction `EJECUTA` without explaining.
2. `MAX_TURNS=5`: the function-calling loop allows up to 5 turns, enough for several-step tasks without falling short.
3. Sequential execution: each tool call is performed, its result is injected into context, and the LLM decides the next step until the task finishes or the turn budget runs out.
4. No intermediate human intervention: the model executes directly; the user only receives the final summarized result.

Example execution

User: `install WordPress with Docker and MariaDB`

Step 1: Verify Docker is installed and running.

Step 2: Create Docker network and MariaDB container with environment variables.

Step 3: Start WordPress container linked to MariaDB.

Step 4: Show final container and exposed port status.

The agent executes each step via `run_command`, receives the output, and advances automatically. At the end it responds with a summary of what was done.

Benefits

- Resolves composite tasks without fragmenting the user's query.
- Leverages the LLM's reasoning to order dependencies (`first MariaDB, then WordPress`).
- Keeps the loop under control: it can ask for confirmation if it detects a destructive or critical step.

Comparison: Qwen3-4B vs Qwen3-8B

During development we evaluated both the Qwen3-4B and Qwen3-8B models (Q4_K_M quantization) served by llama.cpp.

Model	Speed	Function calling reliability	Verdict
Qwen3-4B	Faster, lower latency	Inconsistent · often emits malformed tool calls, invents non-existent functions, or ignores the provided JSON schema	Discarded
Qwen3-8B	Slightly slower, still responsive	Reliable · consistently produces valid	

`tool\_calls` payloads, respects the schema, and completes multi-step tasks correctly |
Definitive model |

Why we chose Qwen3-8B

- Function calling is the core mechanism of this agent. A tool call that does not match the declared schema breaks the execution loop and forces the user to retry.
- Qwen3-4B was attractive because of its lower resource consumption and faster token generation, but in practice it produced enough errors to make the agent unreliable.
- Qwen3-8B maintains acceptable latency on modest hardware while delivering the stability required for unattended, multi-step SRE tasks.

Current deployment

- The production service points to `:8083` and loads Qwen3-8B Q4_K_M.
- The Qwen3-4B model has already been removed from the server and is no longer available.
- All documentation, tests, and examples assume Qwen3-8B as the backing model.

Requirements

- Python 3.10+
- `requests` (`pip install requests`)
- llama.cpp server running Qwen3-8B at `http://localhost:8083/v1/chat/completions`

Usage

python3 chat.py

Example queries:

```
> show disk usage
> read /var/log/syslog
> write a backup script at /tmp/backup.sh
> restart nginx
```

Type `salir`, `exit`, or `quit` to finish.

Security

- Before destructive commands the model warns and asks for confirmation.
- `write\_file` blocks system paths (`/etc`, `/boot`, `/sys`, `/proc`, `/dev`).
- The agent does not persist conversational history across sessions.

Files

- `agent.py` · function-calling orchestrator.
- `tools.py` · shell, read, and write tools.
- `chat.py` · terminal chat interface.
- `README.md` · this document.
- `CHANGELOG.md` · change history.
- `docs/ejecutivo.pdf` · executive summary in PDF.

Version history

v2.0 · SRE Agent with native function calling on Qwen3-8B

- Complete repository rewrite.
- Lightweight SRE agent with native function calling via llama.cpp server.
- New tools:
 - ``run_command``: execute Linux shell commands.
 - ``read_file``: read configuration and log files.
 - ``write_file``: write files, blocking critical system paths.
- Spanish conversational support with up to 5 reasoning turns.
- Multi-step planning and execution without intermediate intervention.
- Basic security: warning before destructive commands and blocking of ``/etc``, ``/boot``, ``/sys``, ``/proc``, ``/dev``.
- Interactive CLI in ``chat.py``.
- Complete README.md with roadmap of implemented features.
- Executive PDF v2.0 in ``docs/ejecutivo.pdf``.

License

MIT / Internal use.